

Minuta Clase 04 — Aspectos Avanzados de Programación Funcional

Materia: Paradigmas y Lenguajes de Programación 2026 — UNTDF / IDEI **Tema:** 04 | **Duración:** 120 minutos | **Filminas:** F-01 a F-36

Objetivos de la clase

Código	Objetivo	Filminas clave
OA-1	Comprender first-class functions y HOF como vocabulario fundacional	F-04, F-05
OA-2	Construir pipelines con <code>pipe</code> (TS) y <code>->></code> (Clojure)	F-06, F-07, F-08
OA-3	Aplicar inmutabilidad y transformación declarativa de colecciones	F-09, F-10, F-11, F-12
OA-4	Diferenciar partial application de currying e implementar ambos	F-13 a F-18
OA-5	Modelar errores con <code>Result<T, E></code> y encadenar validaciones	F-19, F-20, F-21
OA-6	Construir middleware composable con HOF	F-22, F-23
OA-7	Implementar recursión de cola con <code>recur</code> y acumuladores	F-24 a F-27
OA-8	Usar memoization, lazy sequences y DSLs data-driven	F-28 a F-31
OA-9	Mapear patrones entre Clojure y TypeScript	F-32
OA-10	Resolver taller integrador completo	F-33

BLOQUE 1 — Funciones de orden superior y composición (35 min)

[F-01] Portada

Tiempo: 1 min | **Tipo:** portada

Guión docente:

- Proyectar la filmina. Decir: “Hoy vamos a construir sobre lo que ya saben de funciones puras e inmutabilidad. El objetivo: que al salir puedan diseñar un pipeline completo de validación y transformación sin variables mutables.”
- No detenerse: la portada es contexto visual, no contenido.

[F-02] Alcance de la clase

Tiempo: 2 min | Tipo: concepto | OA: OA-1

Guión docente:

- Recorrer brevemente los 4 bloques: "35 min de HOF y composición, 35 min de partial/currying y validación web con Result, 30 min de recursión y patrones avanzados, 20 min de taller."
 - Enfatizar la frase clave: "Dos lenguajes, una misma idea: datos inmutables + funciones puras + composición."
 - Preguntar: "¿Alguno usó `map` o `filter` esta semana en algún proyecto personal o laboral?" — genera conexión.
 - **Transición:** "Empecemos delimitando qué NO vamos a ver hoy."
-

[F-03] Qué no cubrimos hoy

Tiempo: 2 min | Tipo: concepto

Guión docente:

- Leer cada punto y justificar brevemente: "Concurrency merece su propio tema; teoría categórica es abstracta para esta etapa; las librerías FP son herramientas, no conceptos."
 - Cerrar con: "En 120 minutos priorizamos comprensión profunda sobre cobertura superficial."
 - **Por qué importa:** Explicitar los límites reduce la ansiedad del alumno y previene scope creep en preguntas.
-

[F-04] ¿Qué son las first-class functions?

Tiempo: 3 min | Tipo: concepto | OA: OA-1

Guión docente:

- Definir: "Una función es un valor de primera clase cuando se puede asignar a una variable, pasar como argumento y devolver como resultado."
- Preguntar: "¿En qué otros lenguajes que conozcan esto NO es posible?" — esperar que mencionen C (parcialmente) o Java antes de lambdas.
- Conectar: "Sin first-class functions no puede existir `map`, `filter` ni `compose` — todo lo que viene después depende de esto."
- Contraste imperativo/funcional: "Imperativo: digo CÓMO iterar. Funcional: digo QUÉ hacer con cada elemento."

Concepto clave para el alumno: Una función es un dato más. Se almacena, se pasa, se devuelve.

[F-05] Anatomía de una HOF

Tiempo: 3 min | Tipo: concepto | OA: OA-1

Guión docente:

- Mostrar la definición: "HOF = función que recibe y/o devuelve otra función."
- Recorrer los 5 patrones fundamentales: `map`, `filter`, `reduce`, `compose`, `pipe`.
- Para cada uno, dar una frase de una línea:
 - `map` → "aplica f a cada elemento"
 - `filter` → "mantiene los que cumplen el predicado"
 - `reduce` → "pliega toda la lista a un único valor"
 - `compose(f, g)(x)` → "ejecuta g primero, luego f"
 - `pipe(f, g)(x)` → "igual pero de izquierda a derecha"
- Enfatizar: "Estos 5 constituyen el vocabulario con el que se construyen TODOS los patrones avanzados que vemos hoy."

Pregunta rápida: "¿`map` es una HOF? ¿Por qué?" — esperar que identifiquen que recibe una función como argumento.

[F-06] `compose` y `pipe` explicados

Tiempo: 3 min | Tipo: concepto | OA: OA-2

Guión docente:

- Mostrar las fórmulas:
 - `compose(f, g, h)(x) === f(g(h(x)))` — derecha a izquierda
 - `pipe(h, g, f)(x) === f(g(h(x)))` — izquierda a derecha
 - Subrayar las reglas de composición:
 1. Cada función recibe el output de la anterior.
 2. Los tipos deben ser compatibles.
 3. El resultado es una NUEVA función (evaluación diferida).
 4. Se puede nombrar: `const processUser = pipe(trim, normalize, validate)`.
 - Conectar con Clojure: "El operador `->>` hace exactamente lo mismo."
 - **Tip pedagógico:** Dibujar en la pizarra el flujo de datos como flechas: `x → h → g → f → resultado`.
-

[F-07] `compose` y `pipe` en TypeScript

Tiempo: 4 min | Tipo: ejemplo-codigo | OA: OA-2

Guión docente:

- Mostrar la implementación de `pipe` :

```
const pipe = <T>(...fns: Array<(x: T) => T>) =>
  (x: T): T => fns.reduce((acc, fn) => fn(acc), x);
```

- Explicar paso a paso: " `pipe` recibe un array de funciones y devuelve una nueva función que aplica cada una en secuencia usando `reduce` ."
- Recorrer el ejemplo `normalizeEmail` :
 - `trim` → quita espacios
 - `lowercase` → minúsculas
 - `addDomain` → agrega dominio si falta
 - Resultado: " ANA " → "ana@empresa.com"
- Enfatizar: "Cada función es independiente y testeable. `normalizeEmail` es reutilizable en otros pipelines."
- **Actividad rápida:** "¿Qué pasa si agrego `removeAccents` al pipeline? ¿Dónde lo pondrían?" — respuesta: entre `trim` y `lowercase` .

Código completo en filmina: ver F-07 de [filminas.md](#).

[F-08] Thread macro en Clojure

Tiempo: 4 min | Tipo: ejemplo-codigo | OA: OA-2

Guión docente:

- Mostrar primero la versión SIN thread macro: `(reduce + (map #(* % %) (filter even? [1 2 3 4 5])))` — preguntar: "¿Es fácil de leer?" — no.
- Mostrar la versión CON `->>` :

```
(->> [1 2 3 4 5]
  (filter even?)      ; → (2 4)
  (map #(* % %))      ; → (4 16)
  (reduce +))         ; → 20
```

- Explicar: " `->>` inserta el resultado anterior como ÚLTIMO argumento de cada forma. Se lee de arriba hacia abajo."
- Mostrar el ejemplo de pedidos: suma de pedidos completados.
- **Comparación directa:** "¿Ven que es exactamente la misma idea que `pipe` en TS? El patrón es idéntico, la sintaxis es distinta."

[F-09] Datos inmutables: el contrato FP

Tiempo: 3 min | Tipo: ejemplo-codigo | OA: OA-3

Guión docente:

- Mostrar primero el antipatrón imperativo: `user.email = user.email.trim()` — mutación directa.
 - Luego la versión funcional con spread: `{ ...u, email: u.email.trim() }` — nuevo objeto.
 - Construir `normalizeUser = pipe(trimUser, lowercaseEmail)`.
 - Enfatizar: "El objeto original NUNCA se modifica. `raw` sigue igual después de llamar a `normalizeUser`."
 - Conectar con React/Vue: "La comparación por referencia es clave para optimizaciones de rendering."
 - **Pregunta:** "¿Qué consecuencia tiene esto en debugging?" — respuesta: historial de estados rastreable.
-

[F-10] `map`, `filter` y `flatMap` en profundidad

Tiempo: 3 min | Tipo: concepto | OA: OA-3

Guión docente:

- Presentar las firmas de tipo de cada operación:
 - `map: Array<A>.map(A → B): Array<B>` — transforma, NO cambia cantidad.
 - `filter: Array<A>.filter(A → boolean): Array<A>` — selecciona, NO cambia tipo.
 - `flatMap: Array<A>.flatMap(A → Array<B>): Array<B>` — transforma Y aplana.
 - Ejemplo para cada una con datos reales de usuarios.
 - Mencionar Clojure: `mapcat` es el equivalente de `flatMap`.
 - **Tip:** "Cuando necesiten `map` + `flat`, usen `flatMap` directamente."
-

[F-11] `flatMap` aplicado: usuarios y permisos

Tiempo: 3 min | Tipo: ejemplo-codigo | OA: OA-3

Guión docente:

- Mostrar el array de usuarios con roles.
 - `users.flatMap(u => u.roles)` → lista plana de todos los roles.
 - `[...new Set(users.flatMap(u => u.roles))]` → roles únicos.
 - Mostrar el equivalente Clojure: `(mapcat :roles users)`.
 - **Pregunta:** "¿Cómo harían esto con `map` + `flat`?" — mostrar que `flatMap` es más conciso.
-

[F-12] `reduce` y `fold` : plegar colecciones

Tiempo: 4 min | Tipo: ejemplo-codigo | OA: OA-3

Guión docente:

- Afirmar: “`reduce` es el patrón más general. `map` y `filter` son casos especiales de `reduce`.”
 - Ejemplo 1: suma de revenues con pipeline `filter → map → reduce`.
 - Ejemplo 2: construir un índice `Record<string, User>` a partir de un array.
 - Mostrar el equivalente Clojure con `->>`.
 - Enfatizar: “`reduce` convierte una colección en CUALQUIER tipo: arrays, objetos, strings, números.”
 - **Transición al Bloque 2:** “Ahora que dominamos las transformaciones, veamos cómo CONFIGURAR funciones para reusarlas.”
-

BLOQUE 2 — Aplicación parcial, currying y validación web (35 min)

[F-13] Aplicación parcial: el concepto

Tiempo: 3 min | Tipo: concepto | OA: OA-4

Guión docente:

- Definir: “Partial application = fijar algunos argumentos de una función y obtener una función más específica.”
 - Ejemplo conceptual: `add(a, b) → add5 = partial(add, 5) → add5(3) = 8`.
 - Conectar con web: “Piensen en fábricas: `makeValidator('email')` produce un validador especializado. `makeLogger('auth')` produce un logger con prefijo.”
 - **No ejecuta todavía:** “La función parcial NO se ejecuta — espera los argumentos que faltan.”
-

[F-14] Partial application en TypeScript

Tiempo: 3 min | Tipo: ejemplo-codigo | OA: OA-4

Guión docente:

- Mostrar partial manual con closure: `const add5 = (b: number) => add(5, b)`.
 - Mostrar utility general `partial`.
 - Caso real: `makeRequiredValidator` que recibe `fieldName` y devuelve un validator.
 - Crear `validateName`, `validateEmail` — listos para usar en cualquier formulario.
 - Enfatizar: “Cada validator es stateless. Se puede testear con un simple `expect(validateName('')).toEqual(...)`.”
-

[F-15] Partial application en Clojure

Tiempo: 3 min | Tipo: ejemplo-codigo | OA: OA-4

Guión docente:

- Mostrar `partial` nativo: `(def double (partial multiply 2))`.
 - `(map double [1 2 3 4]) → (2 4 6 8)`.
 - Validador: `(def validate-name (partial required-field "nombre"))`.
 - Comparar: "En TS usamos closures; en Clojure existe `partial` como función del lenguaje. Mismo resultado."
-

[F-16] Currying: concepto y diferencia con partial

Tiempo: 3 min | Tipo: concepto | OA: OA-4

Guión docente:

- Definir: "Currying = transformar una función de N argumentos en N funciones anidadas de 1 argumento."
 - `f(a, b, c) → a → b → c → resultado`.
 - Mostrar la TABLA de diferencias (es la filmina más importante para distinguir ambos conceptos):
 - Partial → fija ALGUNOS args → en el momento de usar
 - Currying → convierte en cadena de 1-arg → al definir
 - Conectar: "Currying habilita composición cuando `pipe/compose` necesita funciones de 1 argumento."
 - **Pregunta clave:** "Si tengo `add(a, b)` y hago `curriedAdd(5)(3)`, ¿es currying o partial?" — es currying (cadena de 1-arg). Si fuera `partial(add, 5)(3)` es partial (fijo un arg).
-

[F-17] Currying en TypeScript: implementación

Tiempo: 3 min | Tipo: ejemplo-codigo | OA: OA-4

Guión docente:

- Mostrar `curry2`:

```
const curry2 = <A, B, C>(fn: (a: A, b: B) => C) =>
  (a: A) => (b: B): C => fn(a, b);
```

- Uso: `cHasMinLength(3)` para nombre, `cHasMinLength(8)` para contraseña.
 - Pipeline: `validatePassword = pipe(trim, ...)` usando la versión currificada.
 - Enfatizar: "El tipado de TypeScript infiere correctamente los tipos en cada paso."
-

[F-18] Currying en Clojure: estilo HOF

Tiempo: 3 min | Tipo: ejemplo-codigo | OA: OA-4

Guión docente:

- Mostrar `make-validator` con lambdas anidadas.
 - Validators: `validate-email`, `validate-not-empty`.
 - Clave: `validate-field` con `reduce` — “Si el result es `:ok`, sigue validando; si es `:error`, propaga.”
 - Esto anticipa exactamente el `chain` que veremos en F-20 para TypeScript.
-

[F-19] El tipo `Result<T, E>`

Tiempo: 3 min | Tipo: concepto | OA: OA-5

Guión docente:

- Presentar el tipo:

```
type Result<T, E> =  
  | { status: "ok"; value: T }  
  | { status: "error"; error: E };
```

- Ventajas sobre try/catch (leer de la filmina):
 1. El tipo de retorno hace explícito que puede fallar.
 2. El compilador obliga a manejar ambos casos.
 3. Es composable con `flatMap` / `andThen`.
 4. Más fácil de testear.
 - Corolario: “Si una función puede fallar, su tipo de retorno DEBE decirlo.”
 - **Conectar con Tema 05:** “Este `Result` es exactamente la mónada `Either` que veremos en profundidad.”
-

[F-20] Validación encadenada con `Result`

Tiempo: 4 min | Tipo: ejemplo-codigo | OA: OA-5

Guión docente:

- Mostrar `FormData` con name, email, password.
 - Mostrar 3 validators individuales: `requireName`, `requireValidEmail`, `requireStrongPassword`.
 - Enfatizar: “Cada validator hace UNA cosa.”
 - Mostrar `chain`: “Si hay error, propaga. Si no, continúa.”
 - `validateForm` combina todos con `reduce` + `chain`.
 - **Pregunta:** “¿Qué pasa si quiero acumular TODOS los errores en vez de parar en el primero?” — respuesta: cambiar `chain` por una versión que acumule, o usar `allErrors`. Eso lo ven en el TP.
-

[F-21] Usar el `Result` en un handler HTTP

Tiempo: 3 min | Tipo: ejemplo-codigo | OA: OA-5

Guión docente:

- Mostrar `registerHandler` :
 - `validateForm(req.body)` devuelve `Result`.
 - Si error → `res.status(400).json({ error: result.error })` .
 - Si ok → `userService.create(result.value)` .
 - Enfatizar: "No hay try/catch. El flujo es lineal y explícito."
 - "El tipo `Result` actúa como PROTOCOLO entre validación y lógica de negocio."
 - **Pregunta:** "¿Podrían usar este mismo patrón para validar query params de una API?" — sí, es genérico.
-

[F-22] Middleware como HOF

Tiempo: 3 min | Tipo: concepto | OA: OA-6

Guión docente:

- Definir: "Un middleware es una función que RECIBE un handler y DEVUELVE un handler transformado."
 - Escribir en pizarra: `Middleware = (Request → Response) → (Request → Response)` .
 - "Esto es una HOF: recibe y devuelve funciones."
 - Comparar la versión sin FP (`router.post("/register", authCheck, logRequest, ...)`) con la versión con FP (`pipe(authCheck, logRequest, validateSchema)(handler)`).
 - Enfatizar: "Cada middleware tiene una responsabilidad clara; se pueden reordenar, combinar, testear por separado."
-

[F-23] Middleware en TypeScript: implementación

Tiempo: 4 min | Tipo: ejemplo-codigo | OA: OA-6

Guión docente:

- Definir los tipos: `Request` , `Response` , `Handler` , `Middleware` .
 - `withAuth(secret)` → devuelve Middleware. "Es partial application: fija el secreto."
 - `withLogging(prefix)` → devuelve Middleware con logging.
 - Composición: `const secured = pipe(withLogging("auth-route"), withAuth("my-secret"))(baseHandler)` .
 - "La composición con `pipe` aplica middlewares de afuera hacia adentro."
 - **Transición al Bloque 3:** "Ya sabemos componer, configurar y manejar errores. Ahora: ¿qué pasa cuando el dato es grande o la función es recursiva?"
-

BLOQUE 3 — Recursión de cola y patrones avanzados (30 min)

[F-24] Recursión: el problema del stack overflow

Tiempo: 3 min | Tipo: concepto | OA: OA-7

Guión docente:

- Mostrar la visualización de factorial(5) con los frames apilados.
 - "Cada llamada queda PENDIENTE hasta que la interna termine."
 - "Con listas de 10k o 100k elementos → stack overflow."
 - "Los bucles `for` / `while` no tienen este problema porque no acumulan frames."
 - **Pregunta:** "¿Cómo resolvemos esto sin abandonar la recursión?" → "Con un acumulador."
-

[F-25] Recursión de cola: la idea

Tiempo: 3 min | Tipo: concepto | OA: OA-7

Guión docente:

- Definir: "Una llamada es de cola cuando es la ÚLTIMA operación de la función — no hay trabajo pendiente después."
 - Mostrar la comparación:
 - NO es tail call: `n * factorial(n-1)` — queda multiplicación pendiente.
 - Sí es tail call: `factorial(n-1, acc*n)` — nada pendiente.
 - "Con un acumulador, el runtime puede REEMPLAZAR el frame actual."
 - "Clojure garantiza TCO con `recur` . JavaScript/TypeScript NO garantizan TCO."
 - **Dato práctico:** "V8 (Chrome/Node) no implementa TCO, pero la lógica del acumulador sigue siendo útil para claridad conceptual."
-

[F-26] recur en Clojure

Tiempo: 4 min | Tipo: ejemplo-codigo | OA: OA-7

Guión docente:

- Ejemplo 1: `sum-list` con `recur`:

```
(defn sum-list [nums acc]
  (if (empty? nums)
      acc
      (recur (rest nums) (+ acc (first nums)))))
```

- Trazar paso a paso: `(sum-list [1 2 3] 0)` → `(recur [2 3] 1)` → `(recur [3] 3)` → `(recur [] 6)` → `6`.
 - Ejemplo 2: `my-flatten` — más complejo, con `cond` y `concat`.
 - Enfatizar: “`recur` solo puede llamarse desde posición de cola — Clojure lo VERIFICA en compilación.”
-

[F-27] Recursión de cola en TypeScript

Tiempo: 4 min | Tipo: ejemplo-codigo | OA: OA-7

Guión docente:

- `sumList` con acumulador por default:

```
const sumList = (nums: number[], acc = 0): number =>
  nums.length === 0 ? acc : sumList(nums.slice(1), acc + nums[0]);
```

- `findInTree` con stack explícito — “Cuando el stack puede ser profundo, usar un array como stack.”
 - “La lógica de acumulador es IDÉNTICA a Clojure — el patrón es portable.”
 - Mencionar trampolining como alternativa avanzada (sin implementar).
-

[F-28] Memoization: cache funcional

Tiempo: 3 min | Tipo: concepto | OA: OA-8

Guión docente:

- Definir: “Guardar el resultado para no recalcular con los mismos argumentos.”
 - Cuándo usar: funciones puras con resultados caros, llamadas repetidas con mismos valores.
 - Cuándo NO usar: funciones con efectos colaterales, argumentos de tipo objeto sin serialización.
 - Ejemplo clásico: Fibonacci sin memo $O(2^n)$, con memo $O(n)$.
-

[F-29] Memoization en TypeScript y Clojure

Tiempo: 4 min | Tipo: ejemplo-codigo | OA: OA-8

Guión docente:

- TypeScript: implementación con `Map<T, R>`. Caso real: `getConfig = memoize(parseEnvConfig)`.
 - Clojure: `memoize` nativo. Ejemplo: `fetch-user-cached` con latencia simulada.
 - "Primera llamada ejecuta la función; segunda devuelve desde cache."
 - Enfatizar que la cache solo funciona correctamente con funciones puras.
-

[F-30] Lazy sequences en Clojure

Tiempo: 4 min | Tipo: concepto | OA: OA-8

Guión docente:

- "Una lazy sequence NO calcula sus elementos hasta que se necesitan."
 - `(def naturals (range))` — infinita, no explota memoria.
 - `(take 5 (filter even? naturals))` → `(0 2 4 6 8)`.
 - Ejemplo práctico: pipeline sobre stream de logs — solo los primeros 100 errores.
 - "Sin lazy: cargar todos los logs en memoria. Con lazy: procesar 1 a la vez."
 - Conectar con TS: "Los generators (`function*`) son el equivalente en JavaScript."
-

[F-31] DSLs pequeñas con HOF en Clojure

Tiempo: 5 min | Tipo: ejemplo-codigo | OA: OA-8

Guión docente:

- Mostrar `user-rules` como vector de mapas `{:field :pred :msg}`.
 - Motor genérico `validate` que aplica las reglas a cualquier mapa.
 - "Las reglas son DATOS, no código — se pueden serializar, modificar, extender en runtime."
 - "El motor de validación es genérico — sirve para CUALQUIER entidad."
 - Nombrar el patrón: **data-driven programming**.
 - **Pregunta de cierre del bloque:** "¿Dónde más podrían usar este patrón de reglas como datos?" — configuración, permisos, routing, etc.
 - **Transición:** "Ahora juntemos todo. Mismo problema, dos lenguajes."
-

BLOQUE 4 — Taller y cierre (20 min)

[F-32] Comparación Clojure vs TypeScript

Tiempo: 3 min | Tipo: concepto | OA: OA-9

Guión docente:

- Mostrar la tabla comparativa de 8 patrones.
 - Recorrer RÁPIDO — no leer toda la tabla, agrupar:
 - "Transformaciones: `->>` vs `pipe`, `map` / `filter` / `reduce` — idénticos."
 - "Configuración: `partial` nativo vs closure manual."
 - "Lazy: nativo en Clojure, generators en TS."
 - "TCO: `recur` garantizado vs manual."
 - Cerrar: "Los patrones son TRANSFERIBLES. Si entendés uno, el otro es sintaxis."
-

[F-33] Consigna del taller

Tiempo: 10 min | Tipo: actividad | OA: OA-10

Guión docente:

- Leer la consigna: "API que recibe datos de registro de usuarios. Pipeline funcional sin mutaciones ni try/catch."
 - **Parte A (TypeScript):**
 1. Definir `FormData` con name, email, password, age.
 2. 3 validators con `Result<FormData, string>`.
 3. Componer con `pipe` o `reduce` + `chain`.
 4. Handler HTTP sin excepciones.
 - **Parte B (Clojure):**
 1. `user-rules` como datos.
 2. Motor `validate`.
 3. `memoize` en un lookup.
 4. Pipeline con `->>`.
 - Dar 8 minutos de trabajo. Circular entre grupos. Si alguien se traba, sugerir empezar por los validators individuales.
 - "No importa si no terminan — lo importante es que la ESTRUCTURA del pipeline quede clara."
-

[F-34] Checklist de patrones aprendidos

Tiempo: 2 min | Tipo: resumen

Guión docente:

- Leer la checklist rápidamente — 9 ítems con .
- Pedir que levanten la mano los que se sienten seguros con cada uno.
- Si algún patrón tiene pocas manos → “Ese es buen tema para repasar con la guía de estudio.”

[F-35] ¿Cuándo aplicar estos patrones?

Tiempo: 3 min | Tipo: concepto

Guión docente:

- “La guía práctica: pensar en funcional cuando...” — recorrer los 6 escenarios brevemente.
- “No forzar FP cuando el estado mutable local es claro y acotado.”
- **Pregunta:** “¿Algún ejemplo de sus proyectos donde usarían `Result` en vez de `throw`?”

[F-36] Cierre y nexo con los próximos temas

Tiempo: 2 min | Tipo: resumen

Guión docente:

- 4 puntos clave: funciones como valores, composición como diseño, `Result` como protocolo, recursión de cola.
- Nexo: “Tema 05 = mónadas en TypeScript. El `Result` de hoy es la mónada `Either`.”
- Cita de cierre (Rich Hickey): “Un programa funcional es una colección de transformaciones de datos.”
- “Para la próxima: lean la guía de estudio y empiecen el TP.”

Notas de contingencia

Situación	Acción
El grupo se traba con sintaxis Clojure	Reducir ejemplos Clojure a conceptos y enfocarse en TS
Sobra tiempo en Bloque 1	Agregar ejercicio interactivo de composición
Falta tiempo en Bloque 3	Condensar F-30 y F-31 en explicación oral sin código
El taller (F-33) se queda corto	Extender como ejercicio del TP
Pregunta sobre concurrency	Referir a F-03 y al Tema XX futuro

Distribución temporal total

Bloque	Filminas	Tiempo	Acumulado
Bloque 1: HOF y composición	F-01 a F-12	35 min	35 min
Bloque 2: Partial/currying y validación	F-13 a F-23	35 min	70 min
Bloque 3: Recursión y patrones avanzados	F-24 a F-31	30 min	100 min
Bloque 4: Taller y cierre	F-32 a F-36	20 min	120 min